

Authentication and Access Control Practical

Q – 1 : Create a simple login form for a mock Zomato admin panel that asks for username and password, then displays 'Access Granted' only if both fields match hardcoded admin credentials.

Solution :

```
<h2>Zomato Admin Panel</h2>

<input type="text" id="username" placeholder="Username">
<br><br>

<input type="password" id="password" placeholder="Password">
<br><br>

<button onclick="login()">Login</button>

<p id="result"></p>

<script>
function login() {
  let username = document.getElementById("username").value;
  let password = document.getElementById("password").value;

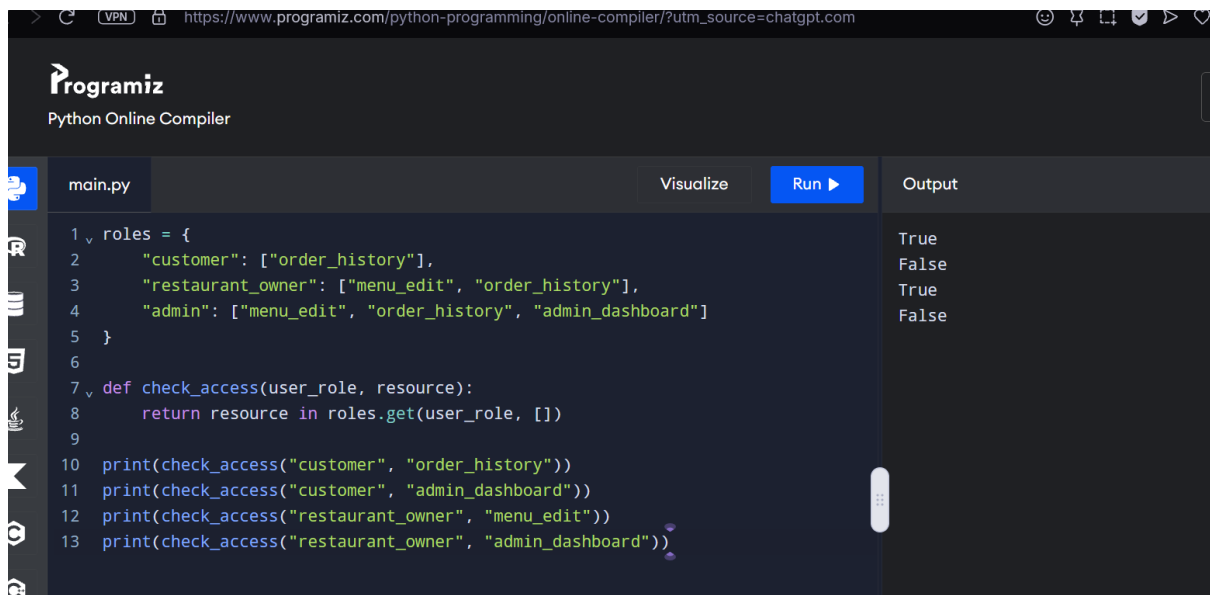
  if (username === "admin" && password === "admin123") {
    document.getElementById("result").innerHTML = "Access Granted";
  } else {
    document.getElementById("result").innerHTML = "Access Denied";
  }
}
```



The screenshot shows a web browser displaying the 'Zomato Admin Panel' login form. The form has a title 'Zomato Admin Panel' at the top. Below the title, there are two input fields: one for the username, which contains the text 'admin', and one for the password, which contains a series of dots. Below the password field is a 'Login' button. Underneath the button, the text 'Access Granted' is displayed, indicating a successful login.

Q – 2 : Write a Python function `check_access(user_role, resource)` that returns `True` if the `user_role` is allowed to access the resource, otherwise `False`. Test it with roles like 'customer', 'restaurant_owner', and resources like 'menu_edit', 'order_history', 'admin_dashboard'. Hint : Use a dictionary to map roles to allowed resources.

Solution :



The screenshot shows the Programiz Python Online Compiler interface. The code editor contains the following Python code:

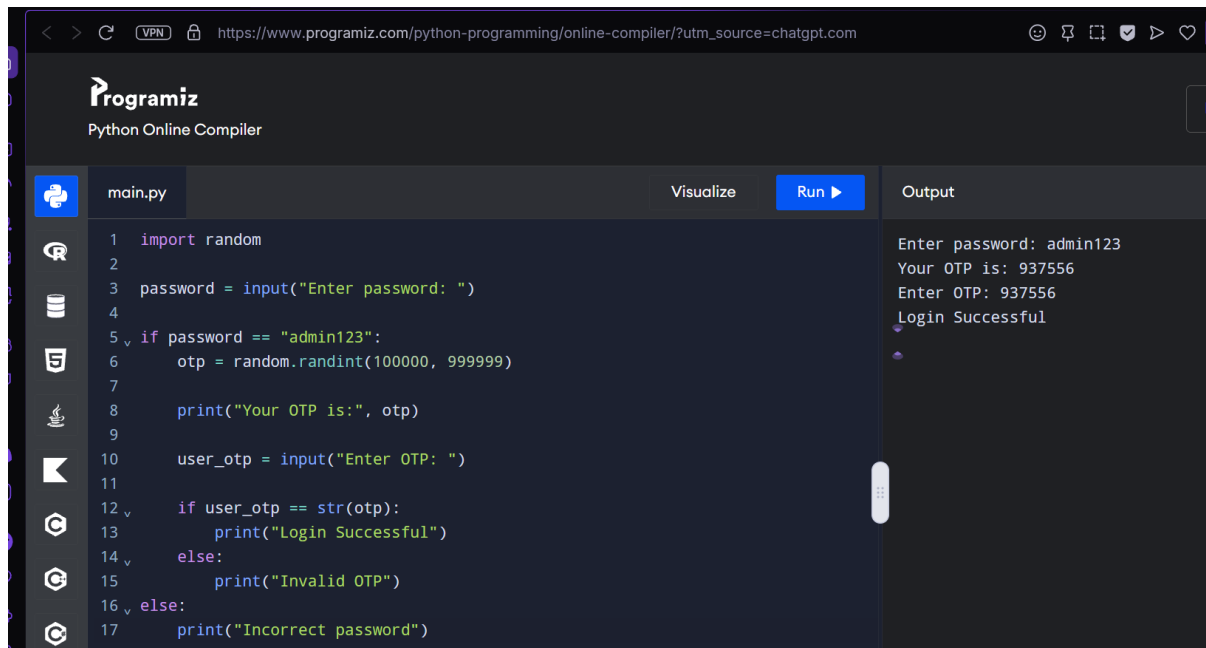
```
1 roles = {  
2     "customer": ["order_history"],  
3     "restaurant_owner": ["menu_edit", "order_history"],  
4     "admin": ["menu_edit", "order_history", "admin_dashboard"]  
5 }  
6  
7 def check_access(user_role, resource):  
8     return resource in roles.get(user_role, [])  
9  
10 print(check_access("customer", "order_history"))  
11 print(check_access("customer", "admin_dashboard"))  
12 print(check_access("restaurant_owner", "menu_edit"))  
13 print(check_access("restaurant_owner", "admin_dashboard"))
```

The output panel on the right shows the results of the function calls:

```
True  
False  
True  
False
```

Q – 3 : Simulate a two-factor authentication flow like Paytm: after a user enters the correct password, generate a 6-digit OTP and display it in the console. Ask the user to input the OTP and display 'Login Successful' only if it matches.

Solution :



The screenshot shows the Programiz Python Online Compiler interface. The code in the editor is as follows:

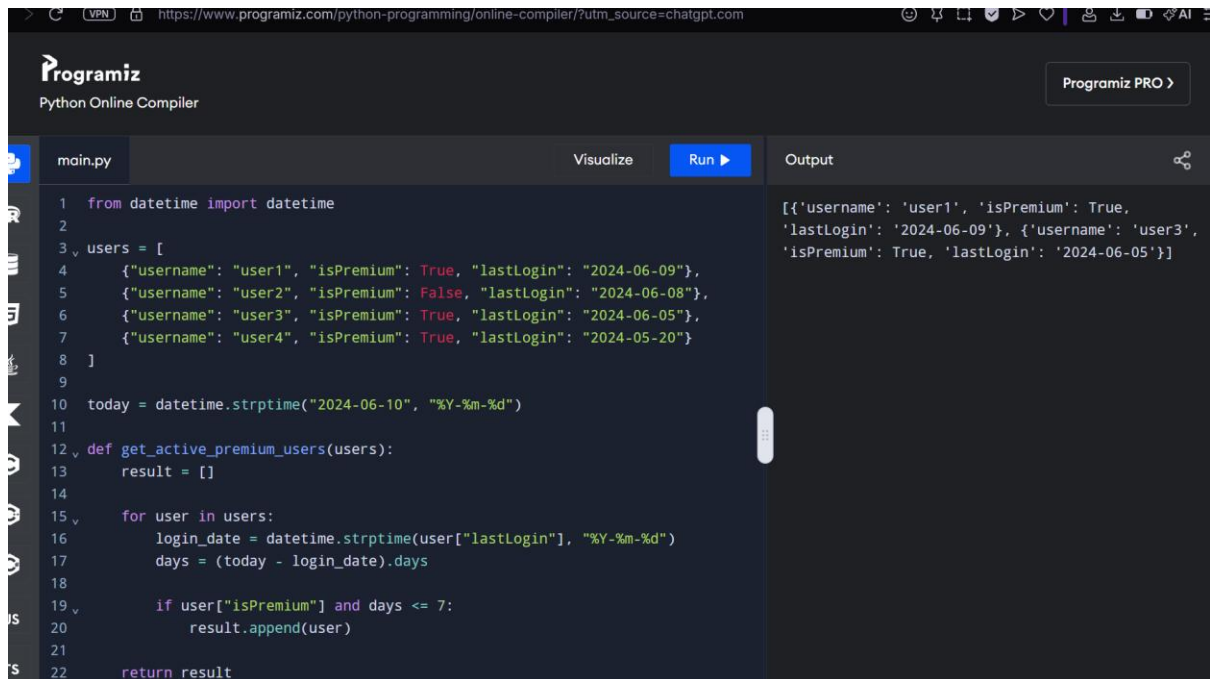
```
1 import random
2
3 password = input("Enter password: ")
4
5 if password == "admin123":
6     otp = random.randint(100000, 999999)
7
8     print("Your OTP is:", otp)
9
10    user_otp = input("Enter OTP: ")
11
12    if user_otp == str(otp):
13        print("Login Successful")
14    else:
15        print("Invalid OTP")
16 else:
17    print("Incorrect password")
```

The output on the right shows the execution results:

```
Enter password: admin123
Your OTP is: 937556
Enter OTP: 937556
Login Successful
```

Q – 4 : Given an array of user objects with properties: username, isPremium, and lastLogin, write a function that returns only those users who are premium and have logged in within the last 7 days. Constraint : Assume lastLogin is a date string in 'YYYY-MM-DD' format, and today is '2024-06-10'.

Solution :



The screenshot shows the Programiz Python Online Compiler interface. The code editor on the left contains a Python script that defines a list of user objects and a function to filter them based on their premium status and login date. The output panel on the right displays the result of the function call.

```
1 from datetime import datetime
2
3 users = [
4     {"username": "user1", "isPremium": True, "lastLogin": "2024-06-09"},
5     {"username": "user2", "isPremium": False, "lastLogin": "2024-06-08"},
6     {"username": "user3", "isPremium": True, "lastLogin": "2024-06-05"},
7     {"username": "user4", "isPremium": True, "lastLogin": "2024-05-20"}
8 ]
9
10 today = datetime.strptime("2024-06-10", "%Y-%m-%d")
11
12 def get_active_premium_users(users):
13     result = []
14
15     for user in users:
16         login_date = datetime.strptime(user["lastLogin"], "%Y-%m-%d")
17         days = (today - login_date).days
18
19         if user["isPremium"] and days <= 7:
20             result.append(user)
21
22     return result
```

The output panel shows the following JSON-like result:

```
[{'username': 'user1', 'isPremium': True, 'lastLogin': '2024-06-09'}, {'username': 'user3', 'isPremium': True, 'lastLogin': '2024-06-05'}]
```

Q – 5 : Use ChatGPT to generate a list of 5 common access control mistakes developers make in real-world apps like Instagram or Flipkart, and for each, write a one-line fix or prevention tip.

Solution : **Five Common Access-Control Mistakes**

No.	Mistake	Prevention
1	Giving users more permissions than required	Follow the principle of least privilege .
2	Checking permissions only on the frontend	Always enforce authorization on the server/backend .
3	Using predictable user/resource IDs	Verify that the logged-in user is authorized to access the requested resource.
4	Forgetting to restrict admin functions	Apply role-based authorization to every administrative function.
5	Not removing old permissions	Review and revoke unnecessary accounts and privileges regularly.